

This proposal was [discussed at this pull request](#).

1 Scoped Thread-Locals

Author: Edmund Noble

Contents

1	Scoped Thread-Locals	1
2	Motivation	1
2.1	Observability and Request Context	1
3	Related Work	2
4	Proposed Change Specification	2
4.1	Library Interface	2
4.2	Primitive Operations	4
4.3	Implementation Notes	4
4.4	Language Design Principle Intersections	4
5	Examples	5
6	Effect and Interactions	6
6.1	Exceptions	6
6.2	Threads	6
6.3	Delimited Continuations	6
6.4	Stack Annotations	6
7	Costs and Drawbacks	6
8	Backward Compatibility	7
9	Alternatives	7
9.1	Explicit context-passing	7
9.2	Reader/ReaderT	7
9.3	Global IORef	7
9.4	Thread-locals via global IORef (Map ThreadId v)	7
9.5	Not inheriting on fork	7
10	Unresolved Questions	7
11	Test Plan	7
12	Implementation Plan	8
13	Endorsements	8
14	Acknowledgments	8

This is a proposal for introducing *scoped thread-locals* into GHC Haskell. Scoped thread-locals provide efficient storage and lookup for immutable context attached to the current Haskell thread. Their primary use cases are observability, request-scoped context, and propagation of context through concurrent programs.

2 Motivation

2.1 Observability and Request Context

Observability tools like loggers and tracers often need to store contextual data during a computation and automatically discard it once that computation finishes.

This proposal was [discussed at this pull request](#).

Consider distributed tracing with OpenTelemetry. A span represents a unit of work and must often be linked to the currently-running span or request context. In Haskell today, that context is usually maintained either explicitly by threading arguments through the program or implicitly via ad-hoc thread-local state stored in library-managed maps. The former is tedious and error-prone; the latter is convenient, but it tends to be awkward around concurrency, because child threads do not inherit the context.

Proposals for heritable, mutable thread-local state have been worked on in the past, and they exist in Java and Racket - but they have a significant performance and semantics issue, the general problem of defensive copies when dealing with mutable state.

GHC already contains specialized mechanisms for attaching information to the execution of a program, such as cost-centre stacks and stack annotations. However, these mechanisms are purpose-built. They are not a general user-facing facility for attaching user-retrievable context to running computations.

Scoped thread-locals address this gap by providing a general-purpose, typed, scope-respecting context mechanism for IO programs.

3 Related Work

Ian Duncan's `thread-utils-context` package provides garbage-collected thread-local storage via `Control.Concurrent.Thread.Storage`. This is useful prior art showing demand for thread-local context in Haskell, but it is based on user-space thread-local maps keyed by thread identity and this state is not heritable between threads.

Ian Duncan's `OpenTelemetry.Context.ThreadLocal` module in `hs-opentelemetry-api` is even closer prior art: it provides per-thread context operations intended for observability and tracing, and its documentation explicitly warns that care is needed around forked threads.

Solonarv's `scoped-values-hs` provides a similar interface to scoped thread-locals, and is implemented using delimited continuations. Scoped values can be inherited across threads if using the provided `forkChild` combinator, which is fairly efficient. However, access to these values requires stack unwinding, which is not as efficient as the proposal included here.

Links:

- `thread-utils-context`: <https://hackage.haskell.org/package/thread-utils-context-0.2.0.0>
- `OpenTelemetry.Context.ThreadLocal`: <https://hackage-content.haskell.org/package/hs-opentelemetry-api-0.3.0.0/docs/OpenTelemetry-Context-ThreadLocal.html>
- `scoped-values-hs`: <https://github.com/Solonarv/scoped-values-hs/>

4 Proposed Change Specification

There are two parts to scoped thread-locals:

- a library interface in `ghc-experimental` and
- a small amount of compiler and RTS support.

4.1 Library Interface

The public API is exposed from `GHC.ScopedThreadLocals.Experimental`, which reexports `GHC.Internal.ScopedThreadLocals`.

The main user-facing interface is based on type-indexed keys. Users allocate keys, and then pass those keys to lookup and binding operations. In the common case, code allocates a key once during initialization and reuses it for all lookups and bindings associated with that logical scoped thread-local. For widely-shared long-lived keys, a top-level definition using `unsafePerformIO` and `NOINLINE` may also be useful; an "immutable global variable".

This proposal was [discussed at this pull request](#).

```
module GHC.ScopedThreadLocals.Experimental where

-- | A runtime key for a scoped thread-local payload type.
data Key v

-- | Allocate a fresh scoped thread-local key.
newKey :: IO (Key v)

-- | A scoped thread-local binding of unknown type.
data SomeScopedThreadLocal
  = forall v. SomeScopedThreadLocal (Key v) v

-- | An immutable snapshot of a thread-local scope.
data ThreadLocalScope

-- | The empty scope.
emptyThreadLocalScope :: ThreadLocalScope

-- | Capture the current thread-local scope.
-- This is intended to be used with `withThreadLocalScope`.
captureThreadLocalScope :: IO ThreadLocalScope

-- | Evaluate a computation with the supplied scope installed for its
-- dynamic extent.
withThreadLocalScope :: ThreadLocalScope -> IO a -> IO a

-- | Look up the current value for a scoped thread-local.
getScopedThreadLocal
  :: Key v
  -> IO (Maybe v)

-- | Install a value for the dynamic extent of an action.
withScopedThreadLocal
  :: Key v
  -> v
  -> IO r
  -> IO r

-- | Effectfully modify the current value for the dynamic extent of an
-- action.
modifyScopedThreadLocal
  :: Key v
  -> (v -> IO v)
  -> IO r
  -> IO r

-- | Strict version of 'modifyScopedThreadLocal'.
modifyScopedThreadLocal'
  :: Key v
  -> (v -> IO v)
  -> IO r
  -> IO r

-- | Effectfully alter the current value for the dynamic extent of an
-- action.
alterScopedThreadLocal
```

This proposal was [discussed at this pull request](#).

```
:: Key v
-> (Maybe v -> IO (Maybe v))
-> IO r
-> IO r

-- | Remove the current value for the dynamic extent of an action.
deleteScopedThreadLocal
  :: Key v
  -> IO r
  -> IO r
```

4.2 Primitive Operations

The low-level interface is:

```
primetype ThreadLocalScope#
{ Opaque representation of the current mapping of scoped thread-locals. }

primop CaptureThreadLocalScopeOp "captureThreadLocalScope#" GenPrimOp
  State# RealWorld -> (# State# RealWorld, (# (# #) | ThreadLocalScope# #) #)
{ Returns the current mapping of scoped thread-locals. }
with
  out_of_line = True
  effect = ReadWriteEffect

primop WithThreadLocalScopeOp "withThreadLocalScope#" GenPrimOp
  ThreadLocalScope#
  -> (State# RealWorld -> (# State# RealWorld, a_reppoly #))
  -> State# RealWorld -> (# State# RealWorld, a_reppoly #)
{ Evaluates the supplied computation with the given scope installed for its
  dynamic extent. The previous scope is restored on normal return and when
  control unwinds past the installed scope. }
with
  out_of_line = True
  effect = ReadWriteEffect
```

4.3 Implementation Notes

The RTS maintains the current thread-local scope as a pointer which is part of a thread's metadata; this pointer points to a single `Map Integer Any`. There is at most one scoped thread-local per runtime key in a scope, so inserting a binding for an already-present key overwrites the current value for that key analogously to shadowing. Changing a thread-local binding is a three-step process. First, a new thread-local scope is constructed out of the previous thread-local scope with the binding inserted, updated, or deleted. Next, a special frame is pushed to the stack that contains the previous thread-local scope, so that when control returns past this frame later, the previous scope is restored. Finally, the thread-local scope field of the thread is set equal to the new scope.

Bindings are stored by the `Integer` carried by a `Key v`. Fresh keys are allocated by `newKey`. This gives type-safe lookup at the library layer while allowing the RTS representation to remain untyped internally.

At the internal-library layer, the representation is manipulated by key-indexed helpers such as `insertThreadLocal`, `lookupThreadLocal`, and `threadLocalScopeToList`.

4.4 Language Design Principle Intersections

The Opt-In Principle:

This proposal was [discussed at this pull request](#).

Code that does not use scoped thread-locals should pay as little as possible for their existence. The implementation is intended to add minimal overhead to code that does not install or capture thread-local scopes: just a single extra scope pointer for each thread.

5 Examples

A simple example:

```
main :: IO ()
main = do
  intKey <- newKey
  print =<< getScopedThreadLocal intKey -- Nothing
  withScopedThreadLocal intKey 10 $ do
    print =<< getScopedThreadLocal intKey -- Just 10
  print =<< getScopedThreadLocal intKey -- Nothing
```

An example displaying shadowing:

```
main :: IO ()
main = do
  intKey <- newKey
  withScopedThreadLocal intKey 10 $ do
    print =<< getScopedThreadLocal intKey -- Just 10
    withScopedThreadLocal intKey 20 $ do
      print =<< getScopedThreadLocal intKey -- Just 20
    print =<< getScopedThreadLocal intKey -- Just 10
```

An example of request-scoped logging context:

```
traceKey :: Key [String]
traceKey = unsafePerformIO newKey
{-# NOINLINE traceKey #-}

pushTrace :: String -> IO a -> IO a
pushTrace seg =
  alterScopedThreadLocal traceKey (\m -> pure (Just (seg : maybe [] id m)))

logMsg :: Logger -> String -> IO ()
logMsg logger msg = do
  trace <- maybe [] reverse <$> getScopedThreadLocal traceKey
  Logger.log logger (show trace <> ": " <> msg)

handleRequest :: Logger -> IO ()
handleRequest logger =
  pushTrace "request" $ do
    logMsg logger "start" -- ["request"]: start
    pushTrace "db" $
      logMsg logger "querying" -- ["request", "db"]: querying
```

An example of explicit scope capture, clearing, and reinstallation:

```
main :: IO ()
main = do
  requestKey <- newKey
```

This proposal was [discussed at this pull request](#).

```
withScopedThreadLocal requestKey "req-123" $ do
  saved <- captureThreadLocalScope
  _ <- forkIO $ do
    print =<< getScopedThreadLocal requestKey -- Just "req-123"
    withThreadLocalScope emptyThreadLocalScope $
      print =<< getScopedThreadLocal requestKey -- Nothing
    withThreadLocalScope saved $
      print =<< getScopedThreadLocal requestKey -- Just "req-123"
  pure ()
```

6 Effect and Interactions

6.1 Exceptions

Exception unwinding restores the surrounding thread-local scope. In particular, if an exception exits a `withScopedThreadLocal` region, the bindings installed by that region are no longer visible after unwinding.

6.2 Threads

New threads created by raw thread-spawn APIs such as `forkIO`, `forkOn`, and `forkOS` inherit the thread-local scope of their parents. Code that wants to clear inherited scoped thread-locals for a dynamic region can use `withThreadLocalScope emptyThreadLocalScope`. Explicit capture with `captureThreadLocalScope` remains useful when a scope needs to be reinstalled later or passed through an API boundary.

6.3 Delimited Continuations

Thread-local scope is part of the dynamic control context.

Consequently:

- capturing a continuation captures the thread-local scope visible at the capture point, and
- resuming a captured continuation restores the captured scope.

This proposal does not pitch scoped thread-locals as a general effect-system mechanism. Continuations are relevant here only because the runtime semantics of scoped context must specify how continuation capture and restoration behave.

6.4 Stack Annotations

Stack annotations are a related feature because they involve stack-attached data, however they do not interact meaningfully with scoped thread-locals.

7 Costs and Drawbacks

This feature adds implementation complexity to the RTS and compiler, including new primops, restore-frame handling, and garbage-collector support for the underlying scope representation.

It also introduces one more ambient mechanism for passing context in Haskell programs. This is useful for observability and request context, but it should not become a substitute for explicit parameters in ordinary program logic.

Finally, the proposal chooses that scoped thread-locals be inherited by child threads by default. This opens the possibility that these values are leaked by long-lived threads.

This proposal was [discussed at this pull request](#).

8 Backward Compatibility

No language breakage is expected.

This is a new opt-in API. Existing user-space thread-local libraries continue to work. Code that wants the new semantics may migrate incrementally.

9 Alternatives

9.1 Explicit context-passing

With Haskell being a functional programming language, passing contextual data via function arguments should be the default. However, certain data are cross-cutting and pervasive enough in programs that explicitly plumbing them around is redundant and even error-prone.

9.2 Reader/ReaderT

ReaderT makes function arguments more implicit, while still ensuring that they are eventually present at the type-level; thus they provide ways to parameterize entire fragments of code. Again however, if the data are cross-cutting and pervasive enough that they should be assumed to be present at any point in a program, or are entirely optional, even this extra machinery at the type-level may be redundant, in which case scoped thread-locals may be a good alternative. OpenTelemetry again provides a prototypical example.

9.3 Global IORef

For practical reasons, many Haskell programs contain global mutable references. However, these significantly decrease modularity and reasonability for the programs in question. Often the parts of these programs which access these global mutable references can't even be tested in parallel. It is the author's hope that many global `IORef`'s may be replaced by scoped thread-locals to reduce the amount of mutation in production Haskell programs.

9.4 Thread-locals via global IORef (Map ThreadId v)

Their scope is limited compared to global `IORef`'s, which is an improvement. The underlying `IORef` is still mutable, which comes with reasonability issues. Inheritance between threads requires the cooperation of the thread-forking code. Even with that cooperation, mutation requires defensive copies, which decreases performance.

9.5 Not inheriting on fork

Another option is to make child threads not inherit the parent scope by default, leaving this up to the forking user.

This proposal rejects that as the default. It's assumed that the heritability of scoped thread-locals is a major part of its reason for being, because this is required for convenient use with OpenTelemetry. The raw fork APIs are commonly used to spawn threads whose lifetime is not tied to the creating scope; it's assumed that this is uncommon enough to not be a problem, and it's possible for programmers to explicitly empty the thread-local scope of a thread they create should they so desire.

10 Unresolved Questions

11 Test Plan

The proposal should be considered implemented correctly only if the API and RTS behavior satisfy tests covering at least the following scenarios:

- lookup outside any scope returns `Nothing`,

This proposal was [discussed at this pull request](#).

- nested scopes shadow and then restore outer values for a key,
- multiple keys can coexist, including distinct keys with the same payload type,
- `deleteScopedThreadLocal` and `alterScopedThreadLocal` can clear an installed binding for the dynamic extent of an action,
- `captureThreadLocalScope` and `withThreadLocalScope` allow re-entering a captured scope, and `emptyThreadLocalScope` clears an installed scope,
- exception unwinding restores the surrounding scope,
- captured continuations restore the scope visible at capture time,
- major GC preserves scoped thread-locals stored on ordinary stacks and in captured continuations,
- raw child threads, including `forkOS`, inherit their parents' thread-local scope,
- explicit scope installation and clearing work across `forkIO`, and
- tight loops that repeatedly install scoped thread-locals do not trigger GC crashes or mis-scavenging.

12 Implementation Plan

There is an active MR for this proposal that matches the requirements set forth here. It can be found at https://gitlab.haskell.org/ghc/ghc/-/merge_requests/15589.

13 Endorsements

Ian Duncan has endorsed this proposal.

14 Acknowledgments

Thanks Hecate Kleidukos, chessai, Jose Cardona, Evan Relf, Ian Duncan, Sam Schlesinger, and S. Shuck for their discussion and feedback on the original proposal. Thanks Solonarv for their discussion and implementing `scoped-values-hs` as an alternative.

This implementation owes a lot to Flatt and Dybvig's **Compiler and Runtime Support for Continuation Marks**.